# Retrieval Augmented Generation

All the LLMs have a big limitation that all their knowledge is stored in their weights (parameters). This leads to three major issues: (a) Private Data ⇒ They cannot answer questions about our proprietary documents.

(b) Knowledge Cutoff ⇒ They cannot answer questions about events that happened after their training date.

(c) Hallucinations ⇒ When unsure, LLMs often confidently invent false facts.

There are two ways using which we can overcome this big limitation of LLMs: 1. Fine-Tuning ⇒ This involves retraining the model on new/our data. But it is expensive, slow, requires technical expertise, and must be repeated every time data changes.

2. RAG ⇒ It involves injecting relevant data into the prompt context at inference time. It relies on in-context learning (the ability to learn from examples or context provided in the prompt without updating weights). It is cheaper, allows real-time data updates, and reduces hallucinations by grounding answers in provided text. It has a 4-step architecture:-

1. Indexing: Create the external knowledge base.
   - ⇒ Ingestion: Load data (PDF, Youtube, Web)
   - ⇒ Chunking: Splitting text into smaller pieces
   - ⇒ Embedding: Convert chunks to vectors.
   - ⇒ Storage: Save vectors in a vector database.
2. Retrieval: Find relevant info:
   - ⇒ Convert user query → vector.
   - ⇒ Perform semantic search (cosine similarity) to find top-k chunks.

Scanned with OKEN Scanner

3. Augmentation: Merge the user query + retrieved context into a single prompt.
4. Generation: Pass the augmented prompt to the LLM to generate the final answer.

# Tools

While LLMs are great at reasoning and generating text, but they inherently can't interact with the outside world (like booking a ticket or querying a database). Tools are python functions packaged so that an LLM can understand and interact with them. There are two types of tools available in LangChain:

(i) Built-In tool ⇒ LangChain offers a wide variety of built-in tools for common tasks like DuckDuckGoSearchRun or the ShellTool.

(ii) Custom tool ⇒ Custom tools are used for our specific business logic or internal APIs. It can be created in three different ways:

(a) @tool Decorator :- This is the simplest method. We wrap a Python function and provide type-hinting and a docstring, which the LLM reads to understand the tool's purpose.

(b) StructuredTool (with Pydantic) :- It allows us to enforce strict input schemas using Pydantic models, which is better for production apps. (c) Subclassing BaseTool :- This is the most advanced method, giving us maximum control and allowing for asynchronous tool execution

# Tool Calling & Binding

To use any tool using LangChain, we first have to register our tools using llm.bind_tools([tool1, tool2]). This passes the tool names, descriptions and input schemas to the LLM. This is known as **Tool Binding**. The LLM does not actually execute the tool. It simply outputs the structured request (a tool call) suggesting which tool to use and what arguments to pass.

⇒ Execution Flow :- 1. User sends a query.
2. LLM returns an 'AIMessage' containing 'tool_calls' list.

Scanned with OKEN Scanner

3. The developer (or framework) executes the actual Python function using the suggested argument.
4. The output is packaged into a special ToolMessage and sent back to the LLM so it can generate a final conversation answer.

# Agents

An agent combines an LLM (the reasoning engine) with tools (the action mechanism) to autonomously achieve a high-level goal.

* **The ReAct Pattern** (Reasoning + Acting) It is a specific design pattern used by LLMs in a continuous loop.

(a) Thought :- The agent reasons about what to do next based on the user query.
(b) Action :- The agent decides to call a specific tool.
(c) Observation :- The agent receives the result from the tool.
(This loop repeats until the agent's "Thought" is that it has the final answer.

⇒ **Agent Vs Agent Executor**

- Agent (create_react_agent) ⇒ It is responsible for "thinking". It decides the next action based on the prompt and available tools.
- Agent Executor ⇒ It is responsible for "doing". It runs the while loop, takes the agent's planned action, actually executes the tool, and feeds the observation back to the agent.

Scanned with OKEN Scanner